Memoria de prácticas

Batalla naval/Hundir la flota

23 de abril del 2023



Índice:

1. Introducción **2**

2. Descripción/Diagrama de clases **4**

3. Ficheros **9**

4. Inconvenientes **11**

5. Conclusiones **11**

1. Introducción

Nota: *La práctica la hemos desarrollado con el enunciado del Grupo A, ya que son más los integrantes del grupo los que pertenecen a esa clase (entregaremos una memoria en español y otra en inglés).*

Integrantes del grupo: Claudia Ortiz Roset (Grupo A), Jaime de la Cueva Sánchez (Grupo A), Sandra Jiménez Ramiro (Grupo C).

Resumen del trabajo realizado:

- **Sandra Jiménez Ramiro:**

Creación de clases con sus atributos, constructores, mutadores, accesorios y métodos empleando herencia (uso de la palabra protected en las clases padre y private en las hijas, base para poder acceder a los atributos y funciones del padre, virtual en la función heredada del padre y override en la función de las hijas), abstracción en la clase Barco y polimorfismo en las funciones de los barcos hijos (Patrulla, Submarino, Destructor y Portaaviones).

En el program: creación y manipulación de ficheros “MisRankings.txt” y “PartidasSinAcabar.txt”; Creación de los objetos de cada clase de barco. Creación de un banner y de un menú funcional implementado con una función fuera del main y un while(!fin) que lo imprime en pantalla hasta que el usuario salga del programa.

Un switch case: case 1, que lee lo que contiene el fichero “PartidasSinAcabar.txt” si no hay partidas guardadas muestra un mensaje, con la ayuda de Claudia hemos conseguido que se carguen las partidas guardadas.

Case 2: pregunta el nombre del jugador 1, muestra el mapa vacío y con tierra en las zonas indicadas en el enunciado y pregunta las posiciones de cada barco, junto con el caso de error para que las coordenadas no se salgan del rango del mapa, con la ayuda de Claudia, que automatizó este caso, posicionando los barcos de la UI de forma aleatoria.

Case 3: pregunta el nombre del jugador 1 y jugador 2, muestra el mapa vacío con tierra y pregunta a ambos jugadores la posición de los barcos, junto con el caso de error para que las coordenadas no se salgan del rango del mapa, con la ayuda de Claudia, que logró que los barcos no se sobrepongan, además de hacer los casos de error para que a la hora de insertar un barco y seleccionar la dirección, este no se salga del mapa.

Case 4: lee lo que contiene el fichero “MisRankings.txt”.

Case 5: creación de un bool llamado “fin” que se sentencia en true, además de un mensaje diciendo que estás saliendo del menú.

Case default: mensaje que dice opción insertada no válida.

Una vez juntadas las partes en una única práctica: Implementé una función llamada “IsLand()” que detecta la tierra y devuelve un bool sentenciado en true, hice que en el mapa se imprima una “L” en las coordenadas donde hay tierra y los casos de error, como por ejemplo que no se pueda disparar a una zona con tierra ni posicionar barcos.

(Intenté hacer que al elegir las direcciones de los barcos, no se posicionasen en tierra creando la función IsLand() que devolvía true si detectaba tierra pero no logré hacer que me funcionara así que si intentas posicionar los barcos en las coordenadas exactas donde hay tierra no te dejará, pero si eliges una coordenada sin tierra y a continuación la dirección, el barco se posicionará en tierra)

Creación de la memoria y diagrama de clases (indicando tipo de atributos, tipo de clases, si se usa herencia, etc. Todo ello con la ayuda de Claudia)

- **Claudia Ortiz Roset:**

Creación de las siguientes clases con sus atributos (siendo protegidos si se utiliza herencia), constructores, mutadores, accesores, métodos y, cuando sea necesario, funciones:

Ship: Implementé esta clase utilizando herencia. Es una clase abstracta, por lo tanto, no tiene funciones. Para esta clase y sus clases hijas, lo hice con Sandra.

Patrol Boat, Submarine, Destructor, Aircraft Carriers: Para estas clases hijas, empecé implementando los constructores (el vacío y el heredado -usando :base() -). También creé la función 'ToString' para cada clase hija para que pudiéramos identificar cada barco en el juego de mesa.

BoardGame: Implementación de la clase con los objetos, la lista y las 6 funciones en ella. Creé la función de mapa con Sandra.

Coordinates: Creación de la clase con sus 3 atributos.

Game: Creación de esta clase con sus atributos, objetos, constructores y 2 funciones (una para jugar contra la 'IA' y otra para jugar 1 contra 1).

Players: Creación de esta clase, con sus atributos, objetos, métodos, mutadores, constructores y múltiples funciones.

Player: Creación de esta clase abstracta padre, con sus atributos protegidos, sus constructores y su función abstracta. Para esta clase y sus clases hijas, lo hice con Sandra.

Player 1, Player 2, Player UI: Creación de estas clases hijas, con sus constructores (el vacío y el heredado) y la función de anulación. Para la clase Player UI, hice que todas las variables fueran aleatorias para que pudiera imitar a un jugador de IA.

Rankings: Creación de esta clase, con sus atributos privados, métodos y mutadores públicos, y constructores. Con la ayuda de Sandra, logramos que el caso 4, la parte de Extractrankings() funcionara, porque estaba dando muchos problemas.

En el programa, se crea (Stream Reader y Stream Writer) los archivos "rankings.txt" y "prevgames.txt", con la ayuda de Sandra.

Además, se crea el menú principal con try - catch, por lo que no importa lo que introduzcas, el programa no colapsará. También, con la ayuda de Sandra, creamos el

do-while con la instrucción switch-case. También comenté todo el código y traduje la documentación que Sandra hizo en español a inglés, y agregué algunos detalles y explicaciones que faltaban.

He hecho que en el case 1 se guarde la información de ambos jugadores con sus tableros, si se decide salir de la partida antes de terminarla. He hecho que todo esté en un string que guarde cada jugador y los 4 tableros estén divididos por ‘;’, que la información de cada jugador esté separado por ‘_’, y que cada coordenada de cada tablero se divida por un ‘.’ . Después he intentado revertir el proceso para poder continuar la partida.

También he hecho que no se pueda disparar dos veces en la misma coordenada utilizando una lista de ‘ShotCoordinates’ para comprobar si se ha disparado en esa coordenada o si no. En el caso de que se haya disparado anteriormente, salta un mensaje y pide la coordenada de nuevo.

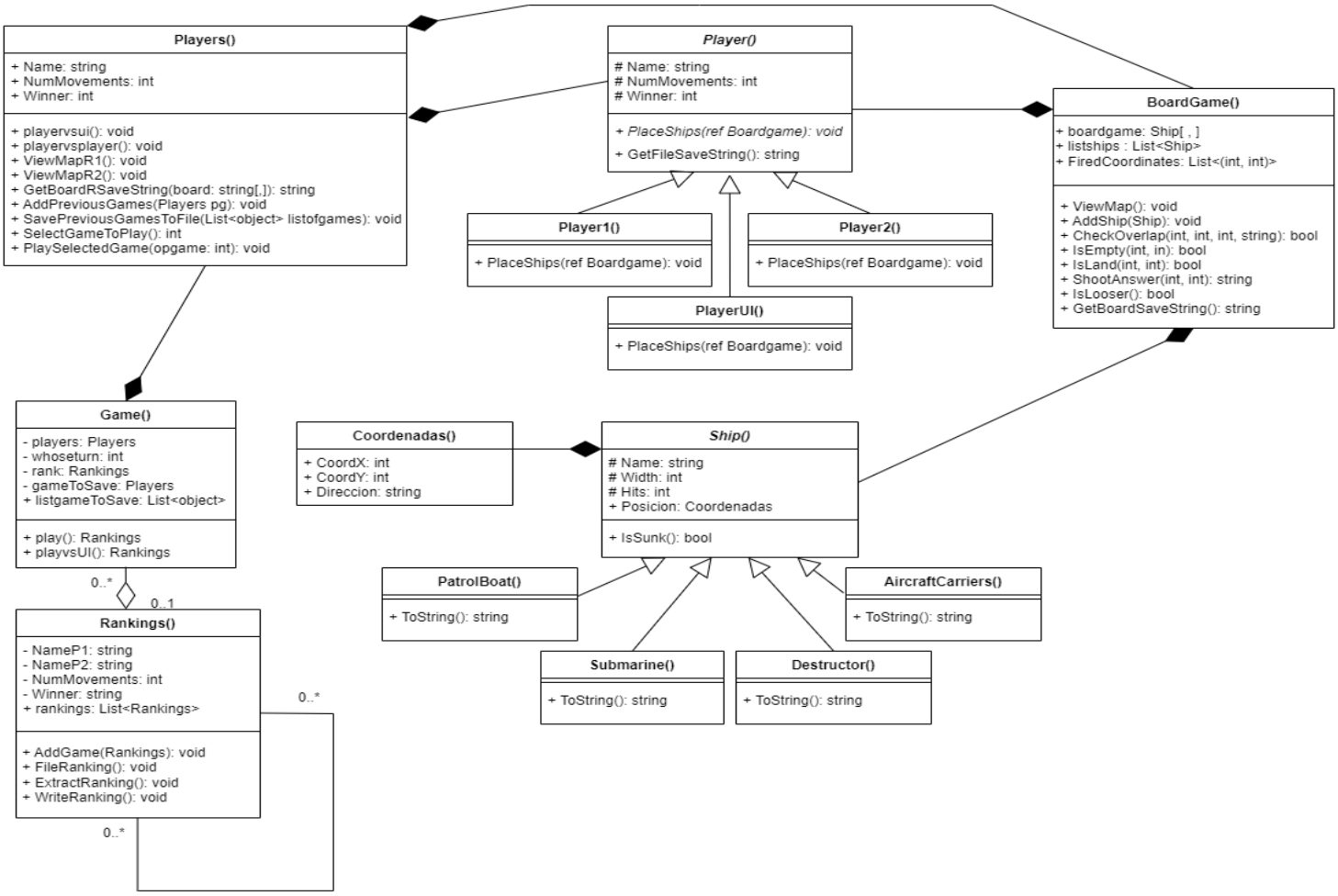
- **Jaime de la Cueva Sánchez:**

Creación de las clases: Ship y sus clases hijas, Player, y Coordenadas, con sus atributos, mutadores, accesores, constructores con sus funciones, en un proyecto aparte para hacer pruebas. Pero como se han hecho después que las de Sandra y Claudia, éstas no se han usado.

En la clase Rankings, he creado sus funciones.

2. Descripción/Diagrama de clases

Descripción general de las clases implementadas y sus decisiones de diseño: (Añadir una imagen sobre el diagrama de clases y sus relaciones).



Clase Players();

· Se trata de una clase pública, la cual tiene tres atributos: public (+) Name que devuelve un string, public (+) NumMovements que devuelve un int y por último public (+) Winner que también devuelve un int.

· Tiene una función *playervsui* que lee el nombre del jugador 1 y lo guarda creando un nuevo objeto Player1, al player 2, le asigna el nombre de la IA. Finalmente llama a las funciones que imprimen los tableros de ambos jugadores (el del P1 y el de la IA preparado para disparar).

· La función playervsplayer es parecida a la anterior, lee los nombres de P1 y P2 e imprime ambos tableros para que cada jugador vaya posicionando sus barcos.

· ViewMapR1 y ViewMapR2 generan ambos tableros que se mostrarán durante el juego, donde el jugador 1 o el jugador 2 (respectivamente) han disparado y si ha dado en un barco o en agua.

· GetBoardSaveString recibe un tablero en formato string[,] y pasa todo el tablero en formato string para poder meterlo en la lista 'listofgames'.

· AddPreviousGames sirve para guardar partidas anteriores y al escribir en el archivo, no se pierda información.

· SavePreviousGamesToFile se encarga de guardar las partidas en el archivo.

· SelectGameToPlay permite seleccionar la partida que deseas cargar de las que hay guardadas.

· PlaySelectedGame te permite seleccionar la partida que quieras cargar.

Clase Player();

· Tiene tres atributos asignados como protected lo que significa que solo se pueden acceder desde dentro de la clase o desde clases derivadas de ella, ya que esta clase abstracta es una clase padre de las clases hijas Player1, Player2 y PlayerUI, estos atributos son: protected (#) Name que devuelve un string, protected (#) NumMovements que devuelve un int y protected (#) Winner que devuelve un int.

· Tiene una única instancia de la función abstracta (puesta en cursiva) llamada PlaceShips, en la cual se pasa el objeto board de la clase BoardGame por referencia. Por lo tanto, sus clases hijas tienen la implementación de la función que guarda la posición de cada barco en el tablero correspondiente al jugador.

· La función GetFileSaveString, lo que hace es guardar lo que hay en el archivo txt en un formato string.

Clase Player1();

· Esta clase hereda de la clase padre Player, por lo tanto indicamos con override que la función que se va a modificar en PlaceShips, a la cual pasamos por referencia el objeto board de la clase BoardGame. Esta función lo que hace es preguntar al usuario las coordenadas de cada barco para posicionarlo en el tablero del jugador correspondiente, además de incluir todos los casos de error de cada coordenada, es decir, si insertas un barco patrulla en la posición (1,1) no podrás direccionarlo ni hacia arriba ni hacia la izquierda, solamente hacia abajo o derecha.

Clase Player2();

· Esta clase también hereda de la clase padre Player (podemos saberlo ya que se usa base para poder acceder a los atributos de Player) y contiene la misma función que Player1, en la que Player2 coloca cada uno de sus barcos.

Clase PlayerUI();

· PlayerUI hereda de la clase Player y posee la misma función que Player1 y Player2, pero en esta función los barcos se colocan de forma aleatoria y controlando estas coordenadas para que se encuentren en el rango permitido.

Clase Ship();

· Sus atributos están asignados con la palabra protected: Name que devuelve un string, Width que devuelve un int y Hits que devuelve un int Coordenadas posicion guarda la posicion de cada barco en la clase Coordenadas.

· Tiene como función IsSunk() que devuelve un bool, esta función comprueba si los disparos ocupan todo el tamaño del barco, si los disparos han hundido el barco devuelve true y si no devuelve false.

Clase PatrolBoat();

· Esta clase hereda de la clase padre Ship, con la palabra base, PatrolBoat puede acceder a los atributos (name, width, hits y posicion) de esta clase padre.

· Tiene una única función llamada ToString() que devuelve una “P”, esta es la letra que aparecerá en el mapa indicando que en esa posición hay un PatrolBoat.

Clase Submarine();

· También hereda de la clase padre Ship. Tiene ToString() como función que devuelve una “S”, letra que aparecerá en el mapa cuando haya un barco submarino.

Clase Destructor();

· Al igual que las anteriores, hereda de la clase Ship. También posee la función ToString() que devuelve una “D”, letra que aparecerá en el mapa cuando haya un barco destructor.

Clase AircraftCarriers();

· Esta clase hereda de la clase Ship. También posee la función ToString() que devuelve una “A”, letra que aparecerá en el mapa cuando haya un barco AircraftCarriers.

Clase Game();

· Sus atributos declarado en private (-) son: private Players players = null, (define una variable players de la clase Players y la inicializa en null) y private whoseturn que devuelve un int.

· Tiene una función play() que no devuelve nada en la que se indica el turno del jugador, su mapa y el mapa del oponente con los disparos que se han realizado con anterioridad, además incluye los casos de error si los disparos se salen del rango del mapa y un mensaje que indica si has disparado al agua o a un barco. Esta función también incluye un do while en el que se le pregunta al usuario si quiere seguir disparando o si desea salir de la partida, si el usuario sigue jugando y gana la partida, saldrá un mensaje indicando que has ganado la partida. Finalmente se guardan los nombres de los jugadores, quien ha sido el ganador y el perdedor y los movimientos que ha realizado cada uno (para mostrarlo en el ranking de partidas).

· La función playvsUI() es la misma que play() pero esta no muestra el mapa de la inteligencia artificial, simplemente en el turno de la UI muestra el disparo que ha realizado sobre el mapa del player 1.

Clase BoardGame();

· Esta clase tiene como atributos un tablero en forma Ship[,], una lista de barcos y otra lista llamada FiredCoordinates del tipo List<(int,int)> que guardará donde se ha disparado para que no se pueda volver a disparar ahí.

· La función ViewMap() se encarga de imprimir el mapa (/tablero).

· A la función AddShip() se le pasa como parámetro el objeto ship de la clase Ship, guarda las coordenadas de cada barco creado y la dirección del mismo, finalmente se añade a la lista ese barco creado.

· A la función CheckOverlap se le pasan como parámetros 3 int (coordenada x, coordenada y, tamaño del barco), y un string (la dirección). Esta función devolvería un void que diría si hay sobreposición de barcos.

· La función IsEmpty comprueba si en las coordenadas que le pases (int coordenada x, int coordenada y) el tablero está vacío o si hay un barco.

· La función IsLand comprueba si en las coordenadas que le pases (int coordenada x, int coordenada y) el tablero tiene tierra o no.

· A la función ShootAnswer() se le pasa como parámetro la coordenada x e y elegida anteriormente por el jugador, si esa coordenada cae en agua, se imprime una “O” en el mapa, en cambio si el disparo coincide con la posición de un barco, se imprime una “X”.

· La función IsLooser() devuelve un bool que será true si el jugador pierde, si de la lista de barcos cada barco creado está hundido, que esto se comprueba con la función IsSunk() que lo que hace es comprobar si los disparos son los mismos que el tamaño del barco, es decir, si el PatrolBoat cuyo tamaño es 2x1, recibe 2 disparos, estará hundido, por lo contrario si haslost es false el jugador habrá ganado.

· La función GetBoardSaveString devuelve un string que se obtiene pasando el tablero a un string para guardarlo en la lista.

Clase Coordenadas();

· Esta clase lo que hace con los mutadores y accesorios es guardar las coordenadas tanto de los barcos como de los disparos.

Clase Rankings();

· En ella se crea una lista de rankings y sus atributos que son: private (-) NameP1 que devuelve un string, private (-) NameP2 que devuelve un string, private (-) NumMovements que devuelve un int y por último private (-) Winner que devuelve un string.

· AddGame lo que hace es añadir los resultados de una partida a la lista de rankings.

· FileRanking sirve para escribir en el archivo los rankings.

· ExtractRanking lee del archivo y guarda en la lista los rankings.

· WriteRanking imprime en pantalla los rankings ordenados de menor a mayor por número de movimientos.

3. Ficheros

Indicar el formato de los ficheros que hemos incluído en la práctica y los datos que contienen:

Para insertar los ficheros en nuestra práctica deberemos añadir el namespace ‘System.IO’ para la correcta manipulación de los ficheros que metamos.

Usando la sintaxis creamos los dos archivos de texto ‘prevgames.txt’ y ‘rankings.txt’.

Con StreamWriter lo abrimos en modo escritura para poder escribir en él y finalmente lo cerraremos con la función .Close();

(Los archivos se guardarán, en este caso en, \bin\Debug).

La primera vez que usemos un fichero será en el caso 1, este fragmento de código tiene la función de leer un archivo de texto "prevgames.txt" y mostrar su contenido por pantalla, línea por línea. Para ello creamos una variable llamada "contador" que se inicializa en 1 y se usa para mostrar el número de línea actual mientras se leen las líneas del archivo.

Creamos un objeto de StreamReader llamado "sr1" para leer el archivo. El archivo se abre en modo de lectura y se maneja dentro de un bloque try-catch para manejar cualquier excepción que pueda ocurrir durante la lectura. El contenido del archivo se lee línea por línea usando el bucle while y se verifica si la línea actual está vacía o no. Si está vacía, se muestra un mensaje de que no hay juegos previos y se sale del bucle. Si no está vacía, se muestra la línea actual. Una vez que se ha leído todo el contenido del archivo, se cierra.

La segunda vez que usemos los ficheros será en el caso 4, tiene la función de leer un archivo de texto llamado "rankings.txt" y mostrar su contenido por consola, línea por línea. Se crea un objeto StreamReader llamado "sr2" para leer el archivo "rankings.txt". El archivo se abre en modo de lectura y se maneja dentro de un bloque

try-catch para manejar cualquier excepción que pueda ocurrir durante la lectura al igual que en el caso 1. El contenido del archivo se lee línea por línea usando el bucle while y se verifica si la línea actual está vacía o no. Si está vacía, se muestra un mensaje de que no hay rankings y se sale del bucle. Si no está vacía, se muestra la línea actual que contiene los rankings. Una vez que se ha leído todo el contenido del archivo, se cierra el objeto de StreamReader y se espera a que el usuario presione cualquier tecla para cerrar la aplicación.

4. Inconvenientes

¿Qué clase de problemas hemos tenido a la hora de realizar la práctica? ¿Cómo los hemos solucionado?

En esta práctica nos han surgido bastantes inconvenientes, por ejemplo a la hora de mostrar las partidas guardadas, lo hemos solucionado de la siguiente manera: Hemos creado una lista de objetos para poder introducir los objetos player1, player2, y los tableros (ya fueran del tipo BoradGame o string[,]). De esta manera, pasamos del player1 y player 2, sus atributos separados por '_' para poder diferenciar los tres atributos de cada jugador. Los 3 atributos los pusimos en modo 'string' para guardarlos en la primera posicion de la lista de objetos, la cual separa todos sus objetos por ';'. Luego, para los tableros, hemos hecho una función para pasar estos tableros de juego y respuesta a modo string para poder poner dicho string en la lista de objetos. Cada coordenada de cada tablero estará separada por '.' . Sin embargo, aunque hemos conseguido guardarlos, no hemos conseguido cargarlos para jugar la partida.

A la hora de posicionar la dirección de los barcos y tener en cuenta el tamaño de cada uno nos han surgido varios tipos de excepciones ya que al situar cualquier barco en ciertas posiciones o direcciones nos daban excepciones y lo hemos solucionado adaptando la clase de PlayerUI como teníamos la de Player1, de esta forma los barcos que se salían de los límites del mapa, se colocaron de forma correcta.

Al querer salir y no continuar disparando el bucle se hacía infinito y no salía de la partida, no permitía guardarla, lo hemos solucionado haciendo que en caso de que el usuario desease salir de la partida, creando un nuevo objeto Player con los datos de la partida, además de guardar la lista con todos los datos del juego en el archivo de texto, también sentenciamos whoseturn a 0 y pusimos un break para salir del juego.

Y finalmente al imprimir los rankings nos ocurría lo mismo que al guardar las partidas, solamente mostraba Player1.GetNombre(), Player1.GetMovements(), Player1.GetWinner() y no lo que debería de salir (Nombre del jugador, cantidad de movimientos y quién es el ganador o el perdedor). Lo solucionamos haciendo una lista de rankings recogiendo los datos de la partida e incluyendo separadores, además de ordenar los rankings según la cantidad de movimientos realizados por el ganador (número de movimientos de mayor a menor) e imprimir el ranking de forma correcta haciendo un foreach en el que si gana Player1 se imprima primero su nombre, movimientos, etc y si el ganador es Player2 lo mismo pero con sus datos.

5. Conclusiones

Conclusiones a las que hayamos llegado realizando la práctica:

A pesar de todos los inconvenientes que hemos podido llegar a tener, hemos sabido cómo actuar en cada momento, trabajando en equipo, que puede llegar a ser muy complicado, ya que no todos hemos tenido los mismos horarios, la misma forma de pensar, etc. A pesar de ello hemos logrado trabajar de una forma colaborativa y con una actitud positiva frente a los problemas que nos iban surgiendo.

Otra conclusión que podemos sacar acerca de la práctica en grupo es que el proceso de realizar la práctica ha sido muy entretenido y nos ha permitido aprender muchas cosas nuevas. No sólo hemos aprendido a saber programar, si no a comunicarnos mejor, resolver problemas y a colaborar en equipo. Todo esto nos será útil en el futuro para afrontar otros retos tanto en nuestra vida académica como en la profesional.

Finalmente algo que nos ha servido de mucho es el poder adaptar la misma práctica del español al inglés, además de tener los integrantes del grupo diferentes perspectivas y formas de programar.